

Sprawozdanie – Laboratoria 4.

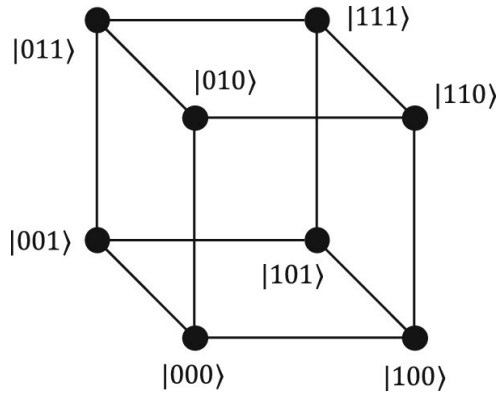
Wojciech Typer

27 stycznia 2026

Zadanie 1

Opis problemu

Wagę Hamminga $H(x)$ definiujemy jako liczbę jedynek dla k -elementowego ciągu bitowego. Ponadto, definiujemy $Z(x)$ jako liczbę zer w tym ciągu. Dla danego $k \in N$ rozważamy k -wymiarową, skierowaną hiperkostkę $H_k = (N_k, A_k)$, to jest graf skierowany o $|N_k| = 2^k$ wierzchołkach, których etykietami są różne ciągi binarne długości k . Krawędzie w hiperkostce H_k łączą wierzchołki etykietowane ciągami różniącymi się na dokładnie jednej pozycji i prowadzą od wierzchołka z etykietą o mniejszej wadze Hamminga do wierzchołka o większej wadze Hamminga. Możemy zauważyć, że każdy wierzchołek $i \in N_k$ jest początkiem lub końcem dokładnie k łuków.



Rysunek 1: Hiperkostka H_3

Pojemność u_{ij} każdej krawędzi $(i, j) \in A_k$ losujemy niezależnie z rozkładem jednostajnym ze zbioru $1, \dots, 2^l$, gdzie $l = \max(H(e(i)), Z(e(i)), H(e(j)), Z(e(j)))$, gdzie etykieta $e(i)$ jest wierzchołkiem $e(i)$ jest k -bitową reprezentacją liczby i

Cel zadanie

Celem zadania jest wyznaczenie maksymalnego przepływu w sieciach, będącymi skierowanymi hiperkostkami H_k dla kolejnych wartości k oraz porównanie czasu działania algorytmu Edmondsa-Karpa i Dinica.

Implementacja

Metoda Forda–Fulkersona

Metoda Forda–Fulkersona jest ogólną procedurą wyznaczania maksymalnego przepływu w sieci przepływowej $G = (V, E)$ z wyróżnionym źródłem s , ujściem t oraz pojemnościami $c(u, v) \geq 0$. Idea metody polega na wielokrotnym znajdowaniu *ścieżek powiększających* w grafie rezydualnym

i zwiększaniu przepływu wzdłuż tych ścieżek, aż do momentu, gdy dalsze powiększanie nie jest możliwe.

Graf rezydualny. Dla aktualnego przepływu f definiuje się pojemność rezydualną

$$c_f(u, v) = c(u, v) - f(u, v),$$

czyli ile dodatkowego przepływu można jeszcze przesłać krawędzią (u, v) . Aby umożliwić modyfikację wcześniej wysłanego przepływu, do grafu rezydualnego dodaje się także krawędzie wsteczne: jeśli $f(u, v) > 0$, to w G_f istnieje krawędź (v, u) o pojemności $c_f(v, u) = f(u, v)$, pozwalająca „cofnąć” część przepływu. Graf rezydualny G_f składa się z tych krawędzi (u, v) , dla których $c_f(u, v) > 0$.

Ścieżka powiększająca i augmentacja. Ścieżka P od s do t w grafie rezydualnym G_f nazywana jest ścieżką powiększającą. Dla takiej ścieżki wyznacza się *wąskie gardło*:

$$\Delta = \min_{(u,v) \in P} c_f(u, v),$$

czyli maksymalną wartość, o jaką można zwiększyć przepływ wzdłuż całej ścieżki, nie przekraczając pojemności. Następnie aktualizuje się przepływ:

- jeśli krawędź (u, v) na ścieżce jest zgodna z kierunkiem oryginalnej krawędzi, zwiększa się $f(u, v)$ o Δ ,
- jeśli wykorzystano krawędź wsteczną (v, u) , to zmniejsza się $f(u, v)$ o Δ (co odpowiada cofnięciu części przepływu).

Po tej operacji zmieniają się pojemności rezydualne i w konsekwencji sam graf rezydualny G_f .

Warunek zakończenia. Procedura jest powtarzana, dopóki istnieje ścieżka powiększająca z s do t w grafie rezydualnym. Gdy t staje się nieosiągalne z s w G_f , nie ma już możliwości powiększenia przepływu i uzyskany przepływ jest maksymalny. Z twierdzenia max-flow min-cut wynika, że brak ścieżki powiększającej jest równoważny istnieniu przekroju o pojemności równej aktualnej wartości przepływu.

Złożoność i uwagi. Metoda Forda–Fulkersona nie precyzuje sposobu wyboru ścieżek powiększających. Dla pojemności całkowitych gwarantuje to zakończenie po skończonej liczbie augmentacji, lecz czas działania zależy od wyboru ścieżek. W skrajnym przypadku liczba iteracji może być bardzo duża. Dwa popularne uszczegółowienia tej metody to:

- Edmonds–Karp: wybiera najkrótszą ścieżkę (BFS), co daje złożoność $O(|V| \cdot |E|^2)$,
- Dinic: pracuje fazami na grafie warstwowym, osiągając $O(|V|^2 \cdot |E|)$ w ogólnym przypadku.

Uwagi implementacyjne. W implementacji praktyczne jest przechowywanie dla każdej krawędzi jej krawędzi odwrotnej (reverse), co umożliwia szybkie aktualizacje pojemności rezydualnych przy augmentacji. Wyszukiwanie ścieżki powiększającej można realizować dowolnym przeszukiwaniem grafu (DFS/BFS), zależnie od wybranej warianty metody.

Edmondsa-Karpa

Algorytm Edmondsa–Karpa jest wariantem metody Forda–Fulkersona wyznaczania maksymalnego przepływu w sieci przepływowej $G = (V, E)$ z wyróżnionym źródłem s i ujściem t oraz pojemnościami $c(u, v) \geq 0$. Kluczową cechą EK jest to, że w każdej iteracji wybiera on ścieżkę powiększającą o najmniejszej liczbie krawędzi, tzn. znajduje ją algorytmem BFS w grafie rezydualnym.

Graf rezydualny. Dla aktualnego przepływu f definiuje się pojemność rezydualną

$$c_f(u, v) = c(u, v) - f(u, v),$$

a także krawędzie wsteczne, które umożliwiają cofanie wcześniej wysłanego przepływu: jeśli $f(u, v) > 0$, to w grafie rezydualnym istnieje krawędź (v, u) o pojemności rezydualnej $c_f(v, u) = f(u, v)$. Zbiór krawędzi rezydualnych tworzy graf G_f .

Iteracja algorytmu. W każdej iteracji:

1. Uruchamiamy BFS w grafie rezydualnym G_f zaczynając od s , aby znaleźć najkrótszą (w liczbie krawędzi) ścieżkę powiększającą do t .
2. Jeśli t jest nieosiągalne, to nie istnieje już ścieżka powiększająca i obecny przepływ jest maksymalny.
3. W przeciwnym razie wyznaczamy *wąskie gardło* ścieżki P :

$$\Delta = \min_{(u,v) \in P} c_f(u, v),$$

a następnie powiększamy przepływ o Δ na krawędziach ścieżki P . Dla krawędzi zgodnych z kierunkiem zwiększamy $f(u, v)$, natomiast dla krawędzi wstecznych zmniejszamy $f(u, v)$ (co odpowiada przesuwaniu przepływu na alternatywne trasy).

Każde takie powiększenie zwiększa wartość przepływu o Δ i jest liczone jako jedna *ścieżka powiększająca* (użyte w eksperymentach jako licznik iteracji algorytmu).

Złożoność. W EK każda iteracja znajduje ścieżkę BFS w czasie $O(|E|)$. Można wykazać, że liczba iteracji jest ograniczona przez $O(|V| \cdot |E|)$, ponieważ odległość (w sensie BFS) najkrótszej ścieżki od s do dowolnego wierzchołka w grafie rezydualnym nie maleje, a „krawędzie krytyczne” (saturujące się na ścieżkach) mogą zmieniać swój status ograniczoną liczbę razy. W konsekwencji całkowita złożoność czasowa wynosi

$$O(|V| \cdot |E|^2).$$

Uwagi implementacyjne. W implementacji wygodnie jest przechowywać sieć w postaci list sąsiedztwa z parami krawędzi: krawędzią *forward* o pojemności rezydualnej oraz krawędzią *reverse* umożliwiającą cofanie przepływu. BFS zapisuje poprzednika wierzchołka i indeks krawędzi, co pozwala po znalezieniu t odtworzyć ścieżkę, wyznaczyć Δ , a następnie zaktualizować pojemności rezydualne na krawędziach forward i reverse.

Algorytm Dinica

Algorytm Dinica (Dinic) służy do wyznaczania maksymalnego przepływu w sieci przepływowej $G = (V, E)$ ze źródłem s , ujściem t oraz pojemnościami $c(u, v) \geq 0$. Podobnie jak w metodzie Forda–Fulkersona operuje on na grafie rezydualnym G_f , lecz przyspiesza działanie dzięki pracy na grafie *warstwowym* oraz wyszukiwaniu *przepływu blokującego*.

Graf rezydualny. Dla aktualnego przepływu f definiuje się pojemności rezydualne $c_f(u, v)$ oraz krawędzie wsteczne o pojemności $f(u, v)$ (umożliwiające cofnięcie części przepływu). Krawędzie o $c_f(u, v) > 0$ tworzą graf rezydualny G_f .

Faza: budowa grafu warstwowego (BFS). W każdej fazie algorytm wykonuje BFS w G_f od s i wyznacza odległości (liczbę krawędzi) do pozostałych wierzchołków. Na tej podstawie definiuje się *graf warstwowy* L , zawierający tylko te krawędzie rezydualne (u, v) , które spełniają warunek:

$$\text{dist}(v) = \text{dist}(u) + 1.$$

W praktyce oznacza to, że w grafie warstwowym przepływ może iść wyłącznie „do przodu” pomiędzy kolejnymi warstwami. Jeśli BFS nie osiąga t (tj. $\text{dist}(t) = -1$), to w G_f nie istnieje ścieżka powiększająca i przepływ jest maksymalny.

Faza: przepływ blokujący (DFS). Mając graf warstwowy L , algorytm wielokrotnie wysyła przepływ z s do t wzdłuż krawędzi warstwowych za pomocą DFS (z ograniczeniem pojemności rezydualnych). Celem jest wyznaczenie *przepływu blokującego* (blocking flow), czyli takiego przepływu w L , po którym żadna ścieżka $s \rightarrow t$ w grafie warstwowym nie ma dodatniej przepustowości rezydualnej (co najmniej jedna krawędź na każdej ścieżce zostaje nasycona). W implementacji stosuje się tablicę wskaźników (np. $\text{it}[v]$), aby nie przeszukiwać w DFS wielokrotnie tych samych krawędzi, które zostały już „wyczerpane” w danej fazie.

Schemat algorytmu. Algorytm powtarza fazy:

1. BFS w G_f i budowa warstw $\text{dist}[\cdot]$.
2. Dopóki istnieje możliwość wysłania dodatniego przepływu w L : DFS zwiększający przepływ (augmentacja) wzdłuż krawędzi spełniających warunek warstwowy.

Po zakończeniu fazy graf warstwowy nie zawiera już ścieżki $s \rightarrow t$ o dodatniej przepustowości; następnie wykonywany jest kolejny BFS, który albo wydłuża najkrótszą odległość do t , albo stwierdza brak ścieżki. To gwarantuje postęp i zakończenie algorytmu z przepływem maksymalnym.

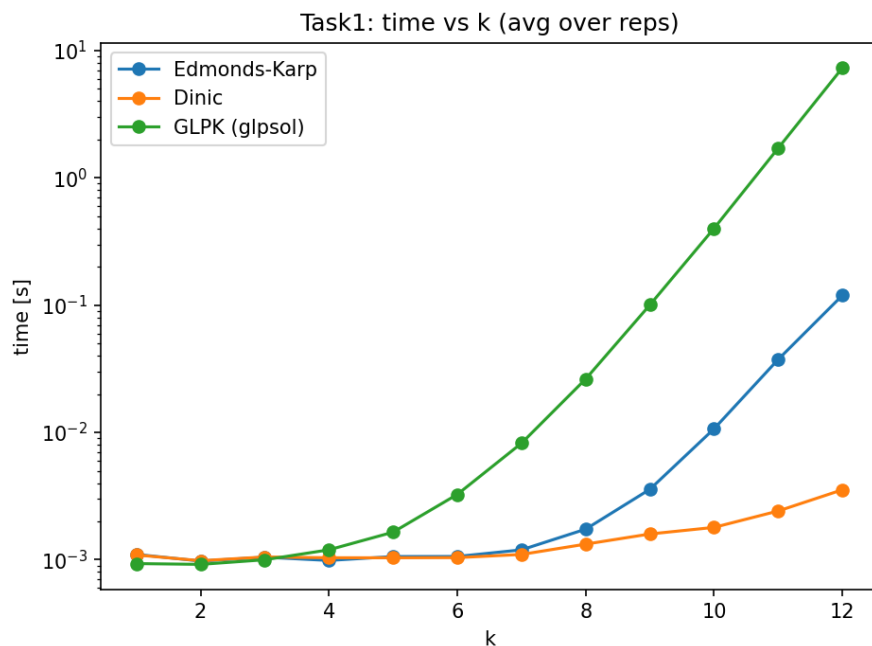
Złożoność. Dla ogólnego grafu skierowanego klasyczna złożoność algorytmu Dinica wynosi

$$O(|V|^2 \cdot |E|).$$

W praktyce jest on zwykle znacząco szybszy niż Edmonds–Karp, ponieważ w jednej fazie (dla ustalonych warstw) potrafi „hurtowo” zwiększyć przepływ do momentu zablokowania wszystkich ścieżek w grafie warstwowym.

Uwagi implementacyjne. Podobnie jak w EK wygodna jest reprezentacja krawędzi w parach (forward/reverse) w liście sąsiedztwa. BFS wyznacza tablicę `level` (odległości warstw), a DFS korzysta z $\text{it}[v]$ (aktualny indeks krawędzi), co zmniejsza liczbę powtórnych przejść po tych samych krawędziach w obrębie jednej fazy. W pomiarach można raportować m.in. liczbę uruchomień BFS (liczba faz) oraz liczbę wywołań DFS (prób wysłania przepływu).

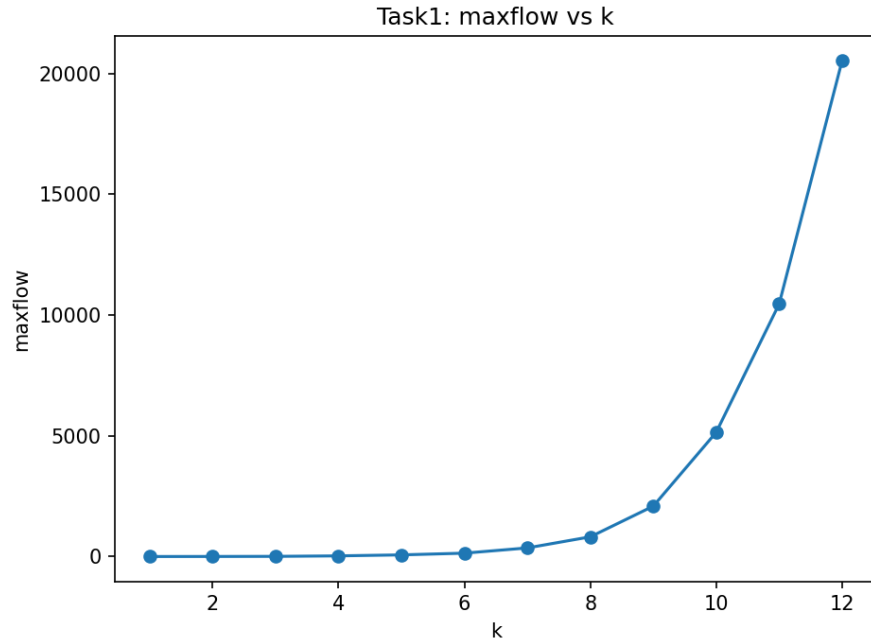
Wyniki



Rysunek 2: Zadanie 1: porównanie czasu działania programów z funkcją k .

Na wykresie powyżej przedstawiono wyniki zależności czasu działania algorytmów Edmonds-Karp'a, Dinic'a od k (w skali logarytmicznej od osi czasu) oraz porównano je z wynikami uzyskanymi przy pomocy GLPK (traktując problem jako programowanie liniowe)

Widać, że wraz ze wzrostem rozmiaru instancji, czasy działania programów rosną. Widać również, że czas pracy algorytmu Dinic'a rośnie najwolniej, natomiast GLPK najszybciej, co oznacza, że jest najmniej optymalny spośród porównanych rozwiązań.



Rysunek 3: Zadanie 1: przykładowa wartość maksymalnego przepływu w funkcji k (dla ustalonego `seed`).

Zadanie 2

Opis problemu

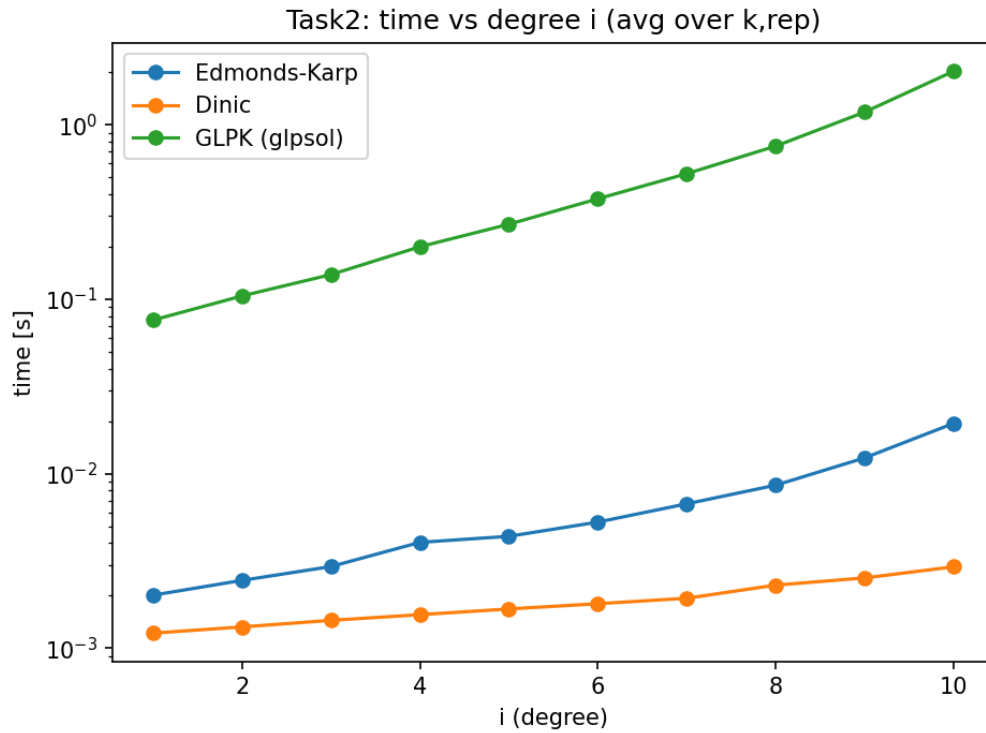
Dla danych $k \in \mathbb{N}$ oraz $i \in \mathbb{N}$ rozważamy nieskierowany dwudzielny graf losowy, mający dwa rozłączne podzbiory wierzchołków V_1 i V_2 , każdy o mocy 2^k . Krawędzie grafu generowane są w taki sposób, że każdy wierzchołek z V_1 ma i sąsiadów z V_2 wybranych niezależnie i jednostajnie losowo.

Cel zadania

Celem zadania jest napisanie programu, który dla podanych parametrów wejściowych, wygeneruje opisany powyżej graf i obliczy wielkość skojarzenia o największym rozmiarze.

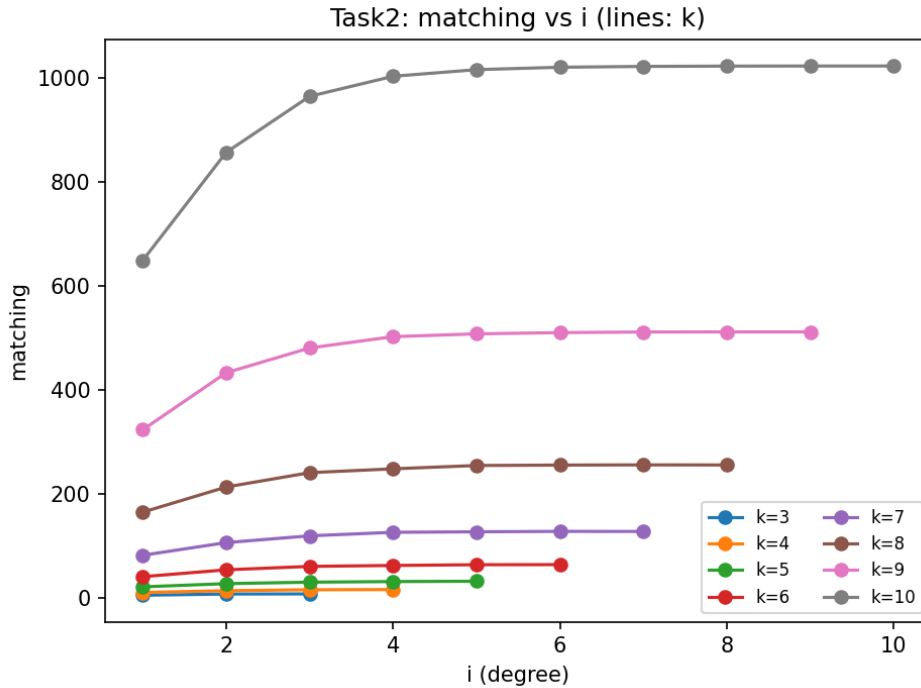
Metodyka. Dla wybranych wartości k oraz stopnia i generowano losowe grafy dwudzielne, używając stałego `seed` w celu powtarzalności. Dla każdej pary parametrów wykonywano kilka uruchomień i uśredniano czasy. Porównano algorytmy Edmonsa–Karpa i Dinica, a dla mniejszych instancji dodatkowo rozwiązanie przez GLPK (poprawność wartości maksymalnego przepływu/matchingu).

Wyniki czasowe. Na rysunku 4 przedstawiono zależność czasu działania od stopnia i (uśrednioną po rozpatrywanych wartościach k oraz powtórzeniach). Oś czasu jest w skali logarytmicznej. Wraz ze wzrostem i graf staje się gęstszy (więcej krawędzi $V_1 \times V_2$), co zwykle zwiększa koszt obliczeń. W praktyce Dinic wypada szybciej od Edmonsa–Karpa dla większych instancji, co jest zgodne z oczekiwaniami dotyczącymi skalowania obu metod.



Rysunek 4: Zadanie 2: porównanie czasu działania w funkcji stopnia i (średnio z kilku uruchomień), oś czasu w skali logarytmicznej.

Wielkość skojarzenia. Rozmiar maksymalnego skojarzenia zależy od parametrów k i i oraz konkretnej realizacji losowania. Dla małych i graf może nie zawierać doskonałego skojarzenia, natomiast przy większym i rośnie szansa na skojarzenie bliskie 2^k . Dla ilustracji na Rysunku 5 pokazano przykładową zależność rozmiaru skojarzenia od i .



Rysunek 5: Zadanie 2: przykładowy rozmiar maksymalnego skojarzenia w funkcji stopnia i (dla wybranych wartości k i ustalonego `seed`).

Poprawność. Dla tych samych instancji (ten sam `seed`) Edmonds–Karp i Dinic zwracały identyczny rozmiar skojarzenia. Dla mniejszych przypadków wynik był dodatkowo zgodny z GLPK, co potwierdza poprawność redukcji matchingu do max-flow oraz implementacji algorytmów.

Podsumowanie. Zadanie 2 pokazuje praktyczne różnice w wydajności algorytmów max-flow na sieciach powstających z grafów dwudzielnych. Wraz ze wzrostem stopnia i rośnie liczba krawędzi w sieci, co wpływa na czas obliczeń. Dinic w praktyce lepiej skaluje się na gęstszych instancjach niż Edmonds–Karp, zachowując tę samą wartość rozwiązania.

Zadanie 3 - Opis modeli programowania liniowego

Model dla Problemu Maksymalnego Przepływu (Zadanie 1)

Model ten opisuje przepływ w skierowanej hiperkostce H_k .

Zmienne decyzyjne:

- $x_{ij} \geq 0$ – wielkość przepływu przez krawędź $(i, j) \in A_k$.

Funkcja celu: Maksymalizacja całkowitego przepływu opuszczającego źródło $s = 0$:

$$\text{maximize } Z = \sum_{(s,j) \in A_k} x_{sj} - \sum_{(j,s) \in A_k} x_{js}$$

Ograniczenia:

- **Ograniczenie pojemności:** Przepływ na każdym łuku nie może przekroczyć wylosowanej pojemności u_{ij} :

$$\forall (i, j) \in A_k : x_{ij} \leq u_{ij}$$

- **Zachowanie przepływu:** Dla każdego wierzchołka poza źródłem s i ujściem $t = 2^k - 1$, suma wpłynięć musi być równa sumie wypłynięć:

$$\forall v \in N_k \setminus \{s, t\} : \sum_{(i,v) \in A_k} x_{iv} - \sum_{(v,j) \in A_k} x_{vj} = 0$$

Model dla Maksymalnego Skojarzenia (Zadanie 2)

Problem maksymalnego skojarzenia w grafie dwudzielnym został sprowadzony do problemu maksymalnego przepływu poprzez dodanie sztucznego źródła S i ujścia T .

Zmienne decyzyjne:

- $x_{ij} \in \{0, 1\}$ – zmienna binarna, informująca czy krawędź należy do skojarzenia.

Funkcja celu: Maksymalizacja sumy krawędzi w skojarzeniu, co odpowiada maksymalnemu przepływowi z S do T :

$$\text{maximize } M = \sum_{(S,u) \in A} x_{Su}$$

Ograniczenia:

- **Ograniczenie stopnia wierzchołków:** Każdy wierzchołek ze zbiorów V_1 i V_2 może być incydentny z maksymalnie jedną krawędzią w skojarzeniu. W modelu przepływowym zapewniają to pojemności łuków ze źródła i do ujścia równe 1:

$$\forall u \in V_1 : x_{Su} \leq 1, \quad \forall v \in V_2 : x_{vT} \leq 1$$

- **Zachowanie przepływu:** Standardowe równania bilansowe dla wierzchołków $u \in V_1$ oraz $v \in V_2$.